

Sonatype Nexus Repository

Модуль для DataOps-инфраструктуры

Цель модуля

В этом модуле вы узнаете, зачем компаниям необходим Sonatype Nexus Repository, как установить и настроить его, а также научитесь загружать и скачивать артефакты. Вы освоите управление артефактами и хранилищем образов в корпоративной DataOps-инфраструктуре.

Содержание модуля:

1. Что такое Sonatype Nexus Repository
2. Установка и настройка Sonatype Nexus
3. Создание пользователей и ролей (для DataOps-команд)
4. Создание репозитория для DataOps
5. Настройка клиентов
6. Рекомендации
7. Использование:
 1. Аутентифицироваться (для Docker – отдельно)
 2. Опубликовать Python-пакет → `pypi-internal`
 3. Установить зависимости через `pypi-all`
 4. Собрать и опубликовать Docker-образ → `docker-data`
 5. Использовать артефакты в CI/CD, Airflow, Kubernetes и т.д.

1. Что такое Sonatype Nexus Repository

Sonatype Nexus Repository – Централизованное хранилище репозитория разнообразных систем управления пакетами (артефактами).

В отличие от систем контроля версий (например, Git), который хранит исходный код – то, что пишут разработчики Nexus Repository хранит готовые файлы (артефакты), которые получаются после сборки этого кода – например, библиотеки, JAR-файлы, Docker-образы и т. п. Вместе они покрывают весь процесс разработки:

– GitHub – для совместной работы над кодом,
– Nexus – для надёжного хранения, управления и защиты готовых компонентов, из которых собирается конечный продукт.

Nexus поддерживает более 20 форматов артефактов как для компонентов с открытым исходным кодом, так и для проприетарных компонентов (<https://help.sonatype.com/en/sonatype-nexus-repository.html>).

Таким образом без центрального репозитория команды рискуют использовать разные версии компонентов, пропустить уязвимости и тратить больше времени на сборку и доставку ПО.

В DataOps-инфраструктуре Sonatype Nexus Repository выходит за рамки классического DevOps-инструмента. Он становится архитектурным элементом доверия, обеспечивающим:

Контроль → Воспроизводимость → Безопасность → Скорость

Без централизованного репозитория команды рискуют:

- использовать несогласованные версии компонентов,
- пропустить уязвимости в зависимостях,
- тратить избыточное время на сборку и доставку ПО.

2. Установка и настройка Sonatype Nexus

Требования

- Установленные Docker и Docker Compose
- Минимум 4 ГБ ОЗУ (рекомендуется 8 ГБ)
- Свободные порты:
 - `8081` — веб-интерфейс
 - `8082-8083` — Docker-репозитории

Развертывание через Docker

Файл `docker-compose.yml`:

```
```yaml
version: '3'
services:
 nexus:
 image: sonatype/nexus3:latest
 container_name: nexus
 restart: unless-stopped
 ports:
 - "8081:8081"
 - "8082:8082"
 - "8083:8083"
 volumes:
 - ./nexus-data:/nexus-data
 environment:
 - INSTALL4J_ADD_VM_PARAMS=-Xms1g -Xmx4g -XX:MaxDirectMemorySize=2g
```
```

Примечание: каталог `/nexus-data` содержит все данные и конфигурации. Для production-сред рекомендуется использовать выделенный том или NFS.

Запуск:

```
```bash
mkdir nexus-setup && cd nexus-setup
Сохраните docker-compose.yml
docker-compose up -d
```
```

Проверка готовности:

```
```bash
docker logs -f nexus
Ожидаемая строка: "Started Sonatype Nexus OSS"
```
```

Веб-интерфейс доступен по адресу: http://localhost:8081

Первоначальная настройка

1. Получите начальный пароль администратора:

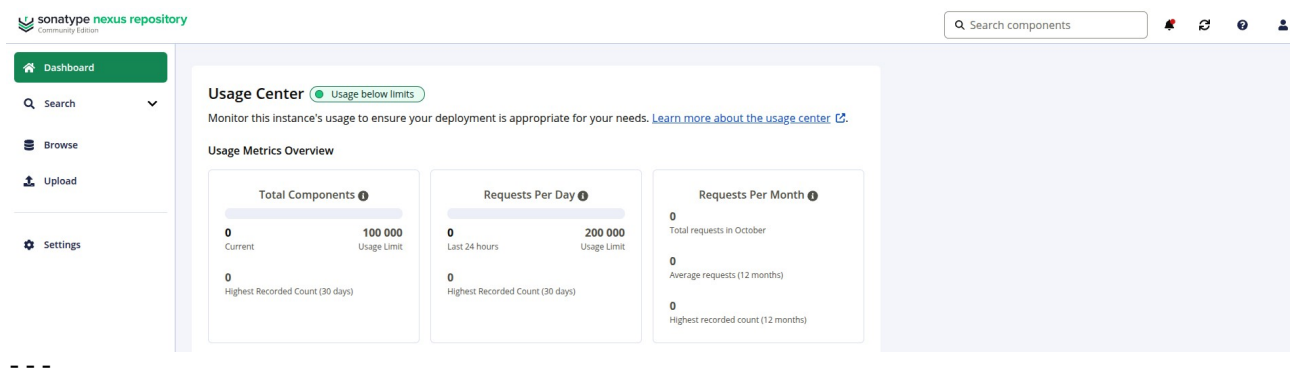
```
```bash
docker exec -it nexus cat /nexus-data/admin.password
```
```

2. Войдите под пользователем `admin`, используя полученный пароль.

3. На первом экране настройки:

- Отключите анонимный доступ (`Enable anonymous access → No`)
- Установите новый надёжный пароль для `admin`
- Сохраните изменения

Важно: анонимный доступ должен быть отключён в production-средах.



3. Создание пользователей и ролей (для DataOps-команд)

В Nexus Repository OSS отсутствует возможность назначать права на конкретные репозитории по имени. Вместо этого используются универсальные wildcard-привилегии, действующие на все репозитории заданного формата.

Пример роли: ``data-scientist-role``

1. Перейдите: Security → Roles → Create role

Settings

Repository

- Repositories
- Blob Stores
- Data Store
- Proprietary Repositories
- Content Selectors
- Cleanup Policies
- Routing Rules

Security

- Privileges
- Roles**

Roles

Manage roles

Create Role

Filter

| ID | NAME | DESCRIPTION |
|---------------------|---------------------|---------------------|
| data-engineer-role | data-engineer-role | |
| data-scientist-role | data-scientist-role | data-scientist-role |
| nx-admin | nx-admin | Administrator Role |
| nx-anonymous | nx-anonymous | Anonymous Role |

2. Укажите:

- Role Type: ``Nexus role``
- Role ID: ``data-scientist-role``

3. В поле Privileges добавьте:

- ``nx-repository-view-pypi-*-edit`` — запись и чтение во все PyPI hosted-репозитории
- ``nx-repository-view-pypi-*-read`` — чтение из всех PyPI проху-репозиториях
- ``nx-repository-view-raw-*-edit`` — запись и чтение во все raw-репозитории (для ML-моделей)

4. Сохраните роль.

Создание пользователя

1. Перейдите: Security → Users → Create local user

The screenshot shows the Sonatype Nexus Repository Community Edition interface. On the left, the 'Settings' sidebar is expanded to 'Security', and the 'Users' link is highlighted. In the main area, the 'Users' page is displayed with a 'Create local user' button at the top. A table lists existing users:

| User ID ↑ | Realm | First name | Last name |
|-----------|---------|---------------|-----------|
| admin | default | Administrator | User |
| anonymous | default | Anonymous | User |

2. Укажите:

- ID: ``ds_user``
- Имя, email, пароль – по усмотрению
- Roles: добавьте ``data-scientist-role``

3. Сохраните.

Для enterprise-сред рекомендуется использовать LDAP/SSO (доступно только в Nexus Repository Pro).

4. Создание репозитория для DataOps

The screenshot shows the Sonatype Nexus Repository Community Edition interface. On the left, the 'Settings' sidebar is expanded to 'Repository'. In the main area, the 'Repositories' page is displayed with a 'Create repository' button at the top. A table lists existing repositories:

| Name ↑ | Type | Format | Blob Store | Status | URL | Health check | Firewall Re... |
|-----------------|--------|--------|------------|---------------------------|-----|--------------|----------------|
| docker-data | hosted | docker | default | Online | | | |
| maven-central | proxy | maven2 | default | Online - Ready to Connect | | | |
| maven-public | group | maven2 | default | Online | | | |
| maven-releases | hosted | maven2 | default | Online | | | |
| maven-snapshots | hosted | maven2 | default | Online | | | |
| ml-models | hosted | raw | default | Online | | | |
| nuget-group | group | nuget | default | Online | | | |
| nuget-hosted | hosted | nuget | default | Online | | | |
| nuget-org-proxy | proxy | nuget | default | Online - Ready to Connect | | | |
| pypi-all | group | pypi | default | Online | | | |
| pypi-internal | hosted | pypi | default | Online | | | |
| pypi-proxy | proxy | pypi | default | Online - Ready to Connect | | | |
| test | hosted | apt | default | Online | | | |
| test2 | hosted | docker | default | Online | | | |

Все репозитории создаются через: Repository → Repositories → Create repository

4.1. PyPI: внутренние Python-пакеты

- Recipe: ``pypi (hosted)``
- Name: ``pypi-internal``
- Deployment policy: ``Allow redeploy`` (dev) / ``Disable redeploy`` (prod)

4.2. PyPI: прокси публичного PyPI

- Recipe: `pypi (proxy)`
- Name: `pypi-proxy`
- Remote storage: `https://pypi.org/simple/`
- Auto-blocking: включить
- Checksum validation: включить

4.3. Групповой PyPI-репозиторий

- Recipe: `pypi (group)`
- Name: `pypi-all`
- Member repositories: `pypi-internal`, `pypi-proxy` (в указанном порядке)
- Единая точка доступа: `http://nexus:8081/repository/pypi-all/simple`

4.4. Docker-репозиторий для аналитических образов

- Recipe: `docker (hosted)`
- Name: `docker-data`
- HTTP port: `8082`
- Force basic authentication: включено

Для работы по HTTP на клиенте необходимо добавить в `/etc/docker/daemon.json`:

```
{ "insecure-registries": ["nexus-host:8082"] }
```

Затем перезапустить Docker: `sudo systemctl restart docker`

4.5. Raw-репозиторий для ML-моделей

- Recipe: `raw (hosted)`
- Name: `ml-models`
- Deployment policy: `Disable redeploy` (для защиты от перезаписи версий)
- Предназначен для хранения файлов: `.pkl`, `.onnx`, `.joblib` и др.

5. Настройка клиентов

Python (pip)

```
``bash
pip install --index-url http://nexus:8081/repository/pypi-all/simple \
            --trusted-host nexus \
            my-data-utils
...`
```

Или в файле `~/.pip/pip.conf`:

```
``ini
[global]
index-url = http://nexus:8081/repository/pypi-all/simple
trusted-host = nexus
...`
```

Docker

```
``bash
docker login nexus:8082
docker tag airflow-etl:1.0 nexus:8082/airflow-etl:1.0
docker push nexus:8082/airflow-etl:1.0
...`
```

Загрузка ML-модели (curl)

```
``bash
Загрузка
curl -u ds_user:password \
      --upload-file model_v1.onnx \
      http://nexus:8081/repository/ml-models/models/prod/model_v1.onnx
...`
```

Скачивание

```
curl -u ds_user:password \
  http://nexus:8081/repository/ml-models/models/prod/model_v1.onnx \
  -o model.onnx
---
```

6. Рекомендации для DataOps

- Резервное копирование: регулярно архивируйте каталог `./nexus-data`
- Безопасность: обновляйте Nexus, используйте HTTPS через reverse proxy (Nginx, Traefik)
- Очистка: настройте политики удаления старых артефактов через Blob store expiration
- Мониторинг: в Pro-версии доступна интеграция с Prometheus и Grafana

Проверка работоспособности

1. В веб-интерфейсе перейдите в раздел Browse
2. Убедитесь, что все созданные репозитории отображаются
3. Загрузите тестовый артефакт (например, `test.txt`) в `ml-models`
4. Авторизуйтесь под созданным пользователем и проверьте доступ к операциям чтения/записи

Использование

Пошаговое описание примеров клиентских операций от имени пользователя `de_user` (Data Engineer) в DataOps-инфраструктуре с использованием Sonatype Nexus Repository OSS.

публикация → использование → развертывание.

3. Примеры клиентских операций от имени `de_user`

Шаг 1. Аутентификация (если требуется явная авторизация)

Для большинства CLI-инструментов (`pip`, `twine`, `docker`, `curl`) аутентификация происходит в момент выполнения команды через передачу логина и пароля. Отдельного «логина» в систему не требуется, за исключением Docker.

Исключение: Docker требует предварительной аутентификации:

```
```bash
docker login nexus:8082 -u de_user -p 'пароль'
```
```

Шаг 2. Публикация внутреннего Python-пакета

Цель: сделать собственную библиотеку (например, `etl_utils`) доступной для других членов команды.

Условия:

- Пакет собран локально (например, через `python -m build`)
- В Nexus создан hosted-репозиторий `pypi-internal`
- У пользователя `de_user` есть роль с привилегией `nx-repository-view-pypi-*`

edit`

Команда:

```
```bash
twine upload --repository-url http://nexus:8081/repository/pypi-internal/ \
 --username de_user \
 --password 'пароль' \
 dist/etl_utils-1.0.0-py3-none-any.whl
```
```

Результат:

Пакет `etl\_utils==1.0.0` становится доступен в `pypi-internal`.

Шаг 3. Установка пакетов (внутренних и внешних)

Цель: использовать как собственные, так и публичные зависимости в проекте.

Условия:

- В Nexus настроен групповой репозиторий `pypi-all`, объединяющий:
 - `pypi-internal` (hosted)
 - `pypi-proxy` (проxy к pypi.org)

Команда:

```
```bash
pip install --index-url http://nexus:8081/repository/pypi-all/simple \
 --trusted-host nexus \
 etl-utils pandas dbt-core
```
```

Как это работает:

1. `pip` запрашивает `etl-utils` → Nexus ищет в `pypi-internal` → находит → отдаёт.
2. `pip` запрашивает `pandas` → Nexus ищет в `pypi-internal` → не находит → идёт в `pypi-proxy` → кэширует и отдаёт.

Преимущество:

Единая точка доступа, ускорение сборок, защита от недоступности pypi.org.

Шаг 4. Сборка и публикация Docker-образа

Цель: развернуть ETL-процесс в изолированной среде (например, в Airflow или Kubernetes).

Условия:

- В Nexus создан hosted-репозиторий `docker-data` на порту `8082`
- На клиентской машине настроен `insecure-registries` (для HTTP)

Последовательность:

1. Сборка образа (локально):

```
```bash
docker build -t airflow-etl:2025.04 .
```
```

2. Аутентификация:

```
```bash
docker login nexus:8082 -u de_user -p 'пароль'
```
```

3. Тегирование под репозиторий Nexus:

```
```bash
docker tag airflow-etl:2025.04 nexus:8082/airflow-etl:2025.04
```
```

4. Публикация:

```
```bash
docker push nexus:8082/airflow-etl:2025.04
```
```

Результат:

Образ доступен для запуска в production-среде:

```
```bash
docker pull nexus:8082/airflow-etl:2025.04
```
```

Шаг 5. Использование артефактов в CI/CD или оркестраторе

Пример: Airflow DAG использует опубликованный образ и пакет.

- DockerOperator в Airflow ссылается на `nexus:8082/airflow-etl:2025.04`
- Внутри контейнера при запуске выполняется:

```
```python
from etl_utils import run_pipeline
run_pipeline()
```
```

Преимущество:

Полная воспроизводимость — тот же код, тот же образ, та же версия зависимостей.

Этот цикл обеспечивает контроль, воспроизводимость и безопасность — ключевые принципы DataOps.